Message Passing Meets Graph Neural Networks: A New Paradigm for Massive MIMO Systems

Hengtao He, *Member, IEEE,* Xianghao Yu, *Member, IEEE,* Jun Zhang, *Fellow, IEEE,* Shenghui Song, *Senior Member, IEEE,* and Khaled B. Letaief, *Fellow, IEEE*

arXiv:2302.06896v2 [cs.IT] 31 Oct 2023

***Abstract*—As one of the core technologies for 5G systems, massive multiple-input multiple-output (MIMO) introduces dramatic capacity improvements along with very high beamforming and spatial multiplexing gains. When developing efficient physical layer algorithms for massive MIMO systems, message passing is one promising candidate owing to its superior performance. However, as their computational complexity increases dramatically with the problem size, the state-of-the-art message passing algorithms cannot be directly applied to future 6G systems, where an exceedingly large number of antennas are expected to be deployed. To address this issue, we propose a model-driven deep learning (DL) framework, namely the AMP-GNN for massive MIMO transceiver design, by considering the *low complexity* of the AMP algorithm and *adaptability* of GNNs. Specifically, the structure of the AMP-GNN network is customized by unfolding the approximate message passing (AMP) algorithm and introducing a graph neural network (GNN) module into it. The permutation equivariance property of AMP-GNN is proved, which enables the AMP-GNN to learn more efficiently and to adapt to different numbers of users. We also reveal the underlying reason why GNNs improve the AMP algorithm from the perspective of expectation propagation, which motivates us to amalgamate various GNNs with different message passing algorithms. In the simulation, we take the massive MIMO detection to exemplify that the proposed AMP-GNN significantly improves the performance of the AMP detector, achieves comparable performance as the state-of-the-art DL-based MIMO detectors, and presents strong robustness to various mismatches.**

***Index Terms*—6G, Bayesian inference, deep learning, graph neural networks, massive MIMO, model-driven.**

I. Introduction

Massive multiple-input multiple-output (MIMO) has been proven as one of the crucial enabling technologies for fifth-generation (5G) systems, where the transmitter/receiver is equipped with tens or even hundreds of antennas to improve the system throughput and spectral efficiency dramatically [2]. It overcomes many challenges such as the massive data traffic and users, large free-space path-loss in millimeter-wave (mmWave) systems, etc. It has been first commercialized with 64-antenna massive MIMO base stations widely deployed by Ericsson, Nokia AirScale, and Huawei [3].

As the adoption of the 5G wireless networks continues to accelerate around the world [4], we are witnessing exciting global research and development activities to formulate the next-generation mobile communication network. This trend is reinforced by the recent emergence of several innovative applications, including the Internet of Everything, Tactile Internet, and seamless virtual and augmented reality [5]. Future wireless networks (6G) are expected to provide ubiquitous coverage, enhanced spectral efficiency (SE), connected intelligence, etc [6]. Facing the new demand, massive MIMO will continuously evolve to ultra-massive MIMO where hundreds or even thousands of antennas are deployed in centralized or distributed manners, and keep playing important roles in future 6G wireless networks [7]. Given the increasing size of antenna arrays, the computational complexity is one of the critical issues when developing efficient physical layer algorithms, such as channel estimation and MIMO detection. In essence, these transceiver design problems can be categorized as *high-dimensional statistical inference* problems and several approaches have been developed accordingly [8].

A. Related Works

The high-dimensional statistical inference problem can be solved by exploiting probabilistic graphical models (PGMs) [9]. Based on the PGMs, many approximate inference algorithms have been developed, including belief propagation (BP) [10], approximated message passing (AMP) [11], and expectation propagation (EP) [12]. These algorithms consider different iterative approaches and have been widely applied to physical layer processing in wireless communications [13], [14]. For instance, iterative detectors based on AMP and EP have been proposed [13], [14]. The AMP-based detector [13] is of low complexity and easy to implement in practice because only the matrix-vector multiplication is involved. In contrast, the EP-based detector [14] achieves Bayes-optimal performance when the channel matrix is unitarily invariant. Nevertheless, it has an extremely higher complexity than the AMP-based detector owing to the required matrix inversion.

Manuscript received February 14, 2023; revised July 14, 2023; accepted September 23, 2023. This paper was presented in part at the IEEE Wireless Commun. Netw. Conf. (WCNC), Glasgow, Scotland, UK, 2023 [1]. This work was supported in part by the Hong Kong Research Grants Council under the Areas of Excellence Scheme Grant AoE/E-601/22-R. The work of Shenghui Song was supported by a grant from the NSFC/RGC Joint Research Scheme sponsored by the Research Grants Council of the Hong Kong Special Administrative Region, China and National Natural Science Foundation of China (Project No. N_HKUST656/22). This work of Jun Zhang was supported in part by the Hong Kong Research Grants Council under Grant 16209622. The work of Xianghao Yu was supported by the Hong Kong Research Grants Council under Grant No. 16212922 and City University of Hong Kong under Project No. 9610629. The associate editor coordinating the review of this paper and approving it for publication was Prof. Pengfei Hu. *(Corresponding author: Xianghao Yu.)*

H. He, J. Zhang, S. Song, and K. B. Letaief are with the Department of Electronic and Computer Engineering, the Hong Kong University of Science and Technology, Hong Kong, E-mail: {eehthe, eejzhang, eeshsong, eekhaled}@ust.hk.

Xianghao Yu is with the Department of Electrical Engineering, City University of Hong Kong (CityU), Hong Kong (e-mail: alex.yu@cityu.edu.hk).

On the other hand, the AMP algorithm has been applied for massive MIMO channel estimation by exploiting the sparsity of the channel observed in the beam domain [15]. A similar idea has been considered by adopting the EP algorithm with better performance but higher complexity [16]. Furthermore, AMP and EP algorithms have been applied to coded linear systems and show asymptotically optimal performance [17], [18]. These applications verify the inherent low-complexity of the AMP algorithm, which is more promising for future wireless communications equipped with large-scale antennas.

Thanks to the strong ability of extracting representative features from data, deep learning (DL) has been recently utilized in the physical layer design of wireless communications [19]–[22], such as mmWave channel estimation [23], [24], channel state information (CSI) feedback [25], and data detection [26]–[29]. Given the interpretability of model-based DL, researchers started to make efforts on improving the message passing-based algorithms in wireless communications by utilizing the model-driven DL technology [21]. Among these works, incorporating the learnable modules into the message passing algorithms is a promising way. For example, convolutional neural network (CNN)-based denoiser has been introduced into the AMP and generalized expectation consistent signal recovery (GEC-SR) algorithms for narrow and wideband beamspace channel estimation, respectively [23], [24]. They both improved the message passing-based algorithms and achieved excellent performance even with a small number of RF chains. Furthermore, the orthogonal AMP (OAMP)-Net and OAMP-Net2 detectors [27] were developed by unfolding the OAMP detector [30] and introducing several learnable parameters. Such detectors were shown to achieve a significant performance improvement compared with the OAMP detector due to the learnable parameters. On the other hand, it has been shown that DL methods can be used to improve a standard BP decoder by assigning learnable weights to the edges in the Tanner graph [31]. Although these works have demonstrated performance improvement by introducing DL, they are trained for fixed MIMO configurations and suffer from poor generalization to handle varying numbers of users or antennas with a single model [31]. This is because these networks have fixed dimensions of the input and output, and lack the inherent structure to adapt to the dynamic dimension of the problem.

To solve this problem, structured neural networks have been introduced to the design of wireless communications. As one of these networks, graph neural networks (GNNs) have attracted much attention in the machine learning field [32]–[34] because of their flexible and adaptive structure. They have been recently adopted in [35]–[39] to exploit the domain knowledge and have been shown to generalize well to different system settings. By incorporating the graph topology of the wireless network into the neural network design, they can improve scalability and generalization. Moreover, GNNs have been applied to learn a message passing solution for statistical inference problems [37], [40]. In particular, a GNN-based MIMO detector was developed by utilizing a pair-wise Markov random field (MRF) model [37]. However, the performance is far worse than existing message passing detectors. More recently, the GEPNet was developed by incorporating the GNN into the EP detector [40], [41]. Although it can achieve the state-of-the-art performance, the computational complexity is prohibitively high because of the matrix inversion in each layer, which is not affordable in future wireless networks with a larger number of antennas. By far, an efficient transceiver design framework, which strikes a better balance between performance and complexity and adapts to dynamic system configurations, is not available.

B. Contributions

To fill this gap, we develop a *low-complexity* model-driven DL-based framework, namely AMP-GNN, which benefits from both the low-complexity of AMP and the adaptability of GNNs. By leveraging the deep unfolding technique [42], [43], we construct the network structure by unfolding the AMP algorithm and incorporating the GNNs module. The main contributions of this work are summarized as follows.

- We first introduce several statistical inference problems and DL-enhanced message passing algorithms in physical layer communications. By incorporating the message passing neural network (MPNN)[1] into the AMP algorithm, we propose a model-driven DL framework, namely AMP-GNN, for massive MIMO transceiver design. In particular, the MPNN module receives the equivalent additive white Gaussian noise (AWGN) observations from AMP as the input and outputs a refined version back to AMP at each layer. Thus, it inherits the low complexity advantages of the AMP algorithm and adaptability from GNNs, which are desirable features for massive MIMO systems.
- We prove that AMP-GNN has the property of permutation equivalence, which is favorable for learning more efficiently, avoiding over-fitting, and developing strong generalizability. Furthermore, we reveal the reason why MPNN can improve the AMP algorithm from the perspective of the EP. This will then provide us with insightful guidelines to combine various GNNs with different message passing algorithms.
- We take the massive MIMO detection as an application and demonstrate that the proposed AMP-GNN-based detector significantly outperforms the existing AMP detector. Furthermore, the AMP-GNN-based detector entails a much lower computational complexity yet comparable performance compared to the state-of-the-art GEPNet detector. Simulation results also show that the AMP-GNN-based detector is robust to channel estimation errors and generalize to different numbers of users with a single model.

Notations—For any matrix $\mathbf{A}$, $\mathbf{A}^T$, $\mathbf{A}^*$, and $\mathrm{tr}(\mathbf{A})$ denote the transpose, conjugate, and trace of $\mathbf{A}$, respectively. In addition, $\mathbf{I}$ is the identity matrix, $\mathbf{0}$ is the zero matrix, and $\mathbf{1}_n$ is the n-dimensional all-ones vector. A proper complex

[1]MPNN is one of the representative GNNs and can unify various GNNs proposed in [33].

TABLE I: Examples of statistical inference problems in physical layer communications.

Problem	Estimated signals $\mathbf{x}$	Measurement matrix $\mathbf{A}$	Measurements $\mathbf{y}$
Channel estimation	Channel	Pilots	Received pilot signals
MIMO detection	Transmitted signals	MIMO channel	Received data signals
Finite-alphabet precoding	Precoding signals	Channel	Transmitted signals
Channel decoding	Codewords	Identity matrix	Detected symbols

Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Omega}$ can be described by the probability density function (pdf):

$$\mathcal{N}_{\mathbb{C}}(\mathbf{z};\boldsymbol{\mu},\boldsymbol{\Omega}) = \frac{1}{\det(\pi\boldsymbol{\Omega})} e^{-(\mathbf{z}-\boldsymbol{\mu})^H \boldsymbol{\Omega}^{-1}(\mathbf{z}-\boldsymbol{\mu})}.$$

The remaining part of this paper is organized as follows. Section II identifies classical statistical inference problems and reviews existing DL-based message passing algorithms. Next, the AMP-GNN is proposed in Section III and several key properties are also analyzed. Section IV elaborates the AMP-GNN for massive MIMO detection. Numerical results are then presented in Section V. Finally, Section VI concludes the paper.

II. Problem Formulation and Algorithms Review

In this section, we first introduce the statistical inference problems in wireless communications in detail. Then, these problems are formulated under the framework of Bayesian inference and the factor graph is elaborated to characterize the inference problem for deriving efficient algorithms. Finally, several existing message passing algorithms are reviewed.

A. Statistical Inference Problems in Wireless Communications

In wireless communication systems, statistical inference is widely utilized in many areas, ranging from transceivers design to network optimization. We consider a standard statistical inference problem that aims to recover the unknown signal $\mathbf{x} \in \mathbb{C}^N$ from a set of measurements $\mathbf{y} \in \mathbb{C}^M$ with the system model

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{n}, \tag{1}$$

where $\mathbf{A} \in \mathbb{C}^{M\times N}$ is the measurement matrix and $\mathbf{n}$ is the additive white Gaussian noise (AWGN). In particular, we list several typical statistical problems in the wireless physical layer in Table I, including channel estimation, MIMO detection, finite-alphabet precoding, and channel decoding. When developing an efficient algorithm to recover signal $\mathbf{x}$, it is assumed that prior knowledge of the underlying signal is known [44]. Such prior knowledge is typically based on the assumption that $\mathbf{x}$ lies in a restricted set $\mathcal{S}$, or follows some distribution with known/unknown parameters. For example, the transmitted symbols $\mathbf{x}$ are the discrete signals chosen from the Q-QAM set $\mathcal{S}$ and the channel is assumed to be Gaussian-mixture distributed with unknown statistical parameters. To solve the statistical inference problem in (1), the Bayesian estimator is considered an efficient approach, which will be introduced in the next subsection.

B. Bayesian Inference and Factor Graph

When solving the statistical inference problem, a powerful approach is to use probabilistic inference under the framework of the Bayesian methodology. According to Bayes' theorem, the posterior probability $\mathsf{P}(\mathbf{x}|\mathbf{y},\mathbf{A})$ can be factorized as

$$\mathsf{P}(\mathbf{x}|\mathbf{y},\mathbf{A}) = \frac{\mathsf{P}(\mathbf{y}|\mathbf{x},\mathbf{A})\mathsf{P}(\mathbf{x})}{\mathsf{P}(\mathbf{y}|\mathbf{A})} = \frac{\mathsf{P}(\mathbf{y}|\mathbf{x},\mathbf{A})\mathsf{P}(\mathbf{x})}{\int \mathsf{P}(\mathbf{y}|\mathbf{x},\mathbf{A})\mathsf{P}(\mathbf{x})\mathrm{d}\mathbf{x}}. \tag{2}$$

Given the posterior probability $\mathsf{P}(\mathbf{x}|\mathbf{y},\mathbf{A})$, the Bayesian MMSE estimate is obtained by

$$\hat{\mathbf{x}} = \int \mathbf{x}\mathsf{P}(\mathbf{x}|\mathbf{y},\mathbf{A})\mathrm{d}\mathbf{x}. \tag{3}$$

However, the Bayesian MMSE estimator is often intractable because the marginal posterior probability for each element in (3) involves a high-dimensional integral, which motivated researchers to develop the approximate inference to obtain the marginal posterior probability effectively. Factor graph, visualizing the dependency on a set of variables with a bipartite graph, is a useful method to develop approximate inference algorithms. It can simplify a joint probability distribution $\mathsf{P}(\mathbf{x}|\mathbf{y},\mathbf{A})$ over many variables $\mathbf{x}$ by factorizing the distribution according to conditional independence relationships.

As illustrated in Fig. 1, the factor graph consists of two kinds of nodes, where the hollow circles represent the variable nodes and the solid squares represent the factor nodes. Based on the factor graph, efficient message passing algorithms for solving inference problems can be obtained by performing different rules. One of the well-known iterative inference algorithms is the BP, which is denoted by the following equations,

$$\mu_{n\to m}^{(t+1)}(x_n) \propto \mathsf{P}(x_n)\prod_{b}^{M}\mu_{b\to n}^{(t)}(x_n), \tag{4a}$$

$$\mu_{m\to n}^{(t)}(x_n) \propto \int \mathsf{P}(y_m|\mathbf{x})\prod_{j\neq n}^{N}\mu_{j\to a}^{(t)}(x_j)\mathrm{d}\mathbf{x}_{\backslash n}, \tag{4b}$$

where the superscript t denotes the index of iteration, $\mathbf{x}_{\backslash n}$ denotes the vector composed by removing the element x_n from $\mathbf{x}$, $\mu_{n\to m}^{(t+1)}(x_n)$ is the message from variable node x_n to factor node $\mathsf{P}(y_m|\mathbf{x})$ at the $(t+1)$-iteration, and $\mu_{n\leftarrow m}^{(t)}(x_n)$ is the message in the opposite direction. Note that the marginal posterior $\mathsf{P}(x_n|\mathbf{y})$ at the t-iteration can be approximated by

$$\hat{\mathsf{P}}^{(t+1)}(x_n|\mathbf{y}) = \frac{\mathsf{P}(x_n)\prod_{m=1}^{M}\mu_{n\leftarrow m}^{(t)}(x_n)}{\int \mathsf{P}(x_n)\prod_{m=1}^{M}\mu_{n\leftarrow m}^{(t)}(x_n)\mathrm{d}x_n}. \tag{5}$$

Thus, the mean of the approximated posterior $\hat{\mathsf{P}}^{(t+1)}(x_n|\mathbf{y})$ can serve as the result of the Bayesian MMSE estimator. Next, we will introduce several classical message passing algorithms and summarize them under a general framework.

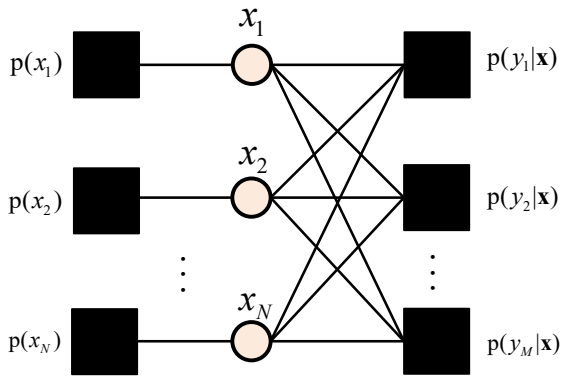


Fig. 1. Factor graph for inverse problems.

C. Existing Message Passing Algorithms

Each iteration of existing message passing algorithms can be represented by the following iterative framework

$$\text{LM}: \quad \mathbf{r}^{(t)} = \gamma_t(\hat{\mathbf{x}}^{(t)}, \hat{\mathbf{v}}^{(t)}), \tag{6a}$$

$$\text{NLM}: \quad \hat{\mathbf{x}}^{(t+1)} = \eta_t(\mathbf{r}^{(t)}, \mathbf{\Sigma}^{(t)}), \tag{6b}$$

where LM and NLM represent the *linear module* γ_t and the *nonlinear module* η_t, respectively. The linear module γ_t takes the current estimate $\hat{\mathbf{x}}^{(t)}$ and associate variance estimate $\hat{\mathbf{v}}^{(t)}$ as inputs, and outputs an intermediate signal $\mathbf{r}^{(t)}$ as well as the associate variance estimate $\mathbf{\Sigma}^{(t)}$. In particular, the linear module is a linear transformation for the input $\hat{\mathbf{v}}^{(t)}$ and $\mathbf{r}^{(t)}$. The function of the linear module is to decouple the linear mixing model (1) into a series of independent equivalent AWGN channels given by

$$r_n^{(t)} = x_n + w_n^{(t)}, \tag{7}$$

where $w_n^{(t)} \sim \mathcal{N}_{\mathbb{C}}(w_n^{(t)}; 0, \Sigma_n^{(t)})$. On the other hand, the nonlinear module η_t takes $\mathbf{r}^{(t)}$ and $\mathbf{\Sigma}^{(t)}$ as inputs, and utilizes denoising operation[2] for $\mathbf{r}^{(t)}$ to obtain a new estimate $\hat{\mathbf{x}}^{(t+1)}$.

In essence, a more powerful linear module can achieve better performance as it can significantly reduce the correlation between different equivalent AWGN channels in (33). For example, because of the LMMSE estimator used in the linear modules in the OAMP and EP algorithms, they can outperform the AMP algorithm. On the other hand, DL-based solutions have been recently shown that it can improve traditional message passing algorithms, such as AMP, OAMP, and EP. As illustrated in Table II, they have been applied to wireless communications with different strategies to tackle different physical layer design problems. Specifically, the OAMP-Net was developed by unfolding the OAMP algorithm and introducing several learnable parameters [27]. The idea is using DL to enhance the performance of the linear module with several scalar learnable variables. Furthermore, the GEPNet, proposed in [40], unfolds the EP algorithm and uses the GNNs to further improve the performance of the LMMSE linear module. However, it has a prohibitively high computational complexity due to the matrix inversion in LMMSE module. As the antenna size is expected to be extremely large in future ultra-massive MIMO systems, it is of paramount importance to adopt a low-complexity linear module, e.g., match filtering (MF), in the message passing algorithm and seek a sophisticated way to further improve the performance of the detector by capitalizing on GNNs.

[2]Denoising operation means the recovery of the signal x_n by removing equivalent noise $w_n^{(t)}$ from (33).

III. Proposed AMP-GNN Network

In this section, we propose an AMP-GNN network for solving statistical inference problems in wireless communications. First, we illustrate the network structure of the proposed AMP-GNN, which is obtained by unfolding the AMP algorithm and incorporating an MPNN module. Then, the AMP algorithm and MPNN module are introduced in detail, respectively. Finally, several key properties of the proposed network and the reasons that lead to its better performance are identified.

A. AMP-GNN Architecture

GNNs have several important advantages, such as modeling interactions between pairs of nodes. We can also exploit them to mitigate the correlation in the decoupled AWGN channels for the AMP algorithm. The block diagram of the AMP-GNN is illustrated in Fig. 2. The network consists of T cascade layers, and each layer has the same structure that contains a GNN module and the conventional AMP algorithm. The input of the AMP-GNN is the received signal $\mathbf{y}$, with the initial value setting as $\hat{\mathbf{x}}^{(1)} = \mathbf{0}$ and $\hat{\mathbf{v}}^{(1)} = \frac{N}{M}\mathbf{1}_N$, and the output is the final estimate $\hat{\mathbf{x}}^{(T)}$ of signal $\mathbf{x}$. For the t-th layer of the AMP-GNN, the inputs are the estimated signal $\hat{\mathbf{x}}^{(t-1)}$ and $\hat{\mathbf{v}}^{(t-1)}$ from the $(t-1)$-th layer and the received signal $\mathbf{y}$. Finally, the AMP-GNN is executed iteratively until terminated by a fixed number of layers. In the next subsection, we introduce the AMP algorithm and the structure of the adopted MPNN module in detail.

B. AMP algorithm

The AMP algorithm was first proposed to solve sparse linear inverse problems in compressed sensing [45], and has been widely used in various scenarios [13], [15], [17]. It admits a rigorous analysis based on state evolution. Such advantages motivated researchers to apply the AMP algorithm in wireless communications. In **Algorithm 1**, we summarize the AMP algorithm for an arbitrary signal[3], where m and n are the indexes of $\mathbf{y}$ and $\mathbf{x}$, respectively. The main principle of the algorithm is to decouple the posterior probability $\mathsf{P}(\mathbf{x}|\mathbf{y}, \mathbf{A})$ into a series of $\mathsf{P}(x_n|\mathbf{y}, \mathbf{A})$, for $n = 1, 2, \ldots, N$, in an iterative way. In particular, $\mathsf{P}(x_n|\mathbf{y}, \mathbf{A})$ is assumed to be a Gaussian distribution that is obtained from the equivalent AWGN model in (33). Equations (8e) and (8f) perform the posterior mean and variance estimation for the equivalent AWGN model (33) and exact expressions are related to the prior information of the signal. If the transmitted symbol is assumed to be drawn

[3]Note that we consider a complex-valued AMP-based MIMO detector in **Algorithm 1** and the equivalent real-valued form can be derived with the equivalent real-valued representation accordingly.

TABLE II: Components of different message passing Algorithms

Algorithms	Linear module	Nonlinear module	Learnable module
AMP [13]	MF	MMSE denoiser	/
EP/OAMP/VAMP [12]	LMMSE	MMSE denoiser	/
OAMP-Net [27]	Learnable LMMSE	Divergence-free denoiser	Linear module
GEPNet [40]	LMMSE+GNN	MMSE denoiser	Linear module
LDAMP [23]	MF	CNN-based denoiser	Nonlinear module
LDGEC [24]	LMMSE	CNN-based denoiser	Nonlinear module

Algorithm 1: AMP algorithm

1. Input: $\mathbf{y}$, $\mathbf{A}$, σ^2, $\mathsf{P}(\mathbf{x})$.
2. Initialization: $\hat{x}_n^{(1)} = 0$, $\hat{v}_n^{(1)} = \frac{N}{M}$, $Z_m^{(0)} = y_m$.
3. Output: $\hat{\mathbf{x}}^{(T)}$.
4. Iteration:
for $t = 1, \cdots, T$ **do**

$$V_m^{(t)} = \sum_{n=1}^{N} |a_{mn}|^2 \hat{v}_n^{(t)} \tag{8a}$$

$$Z_m^{(t)} = \sum_{n=1}^{N} a_{mn}\hat{x}_n^{(t)} - \frac{V_m^{(t)}(y_m - Z_m^{(t-1)})}{\sigma^2 + V_m^{(t-1)}} \tag{8b}$$

$$\Sigma_n^{(t)} = \left(\sum_{m=1}^{M} \frac{|a_{mn}|^2}{\sigma^2 + V_m^{(t)}}\right)^{-1} \tag{8c}$$

$$r_n^{(t)} = \hat{x}_n^{(t)} + \Sigma_n^{(t)} \sum_{m=1}^{M} \frac{a_{mn}^*(y_m - Z_m^{(t)})}{\sigma^2 + V_m^{(t)}} \tag{8d}$$

$$\hat{x}_n^{(t+1)} = \mathbb{E}\{x_n | r_n^{(t)}, \Sigma_n^{(t)}\} \tag{8e}$$

$$\hat{v}_n^{(t+1)} = \mathrm{Var}\{x_n | r_n^{(t)}, \Sigma_n^{(t)}\} \tag{8f}$$

from the Q-QAM set $\mathcal{S} = \{s_1, s_2, \ldots, s_Q\}$, the results in (8e) and (8f) are given by

$$\hat{x}_n^{(t+1)} = \frac{\sum_{s_i \in \mathcal{S}} s_i \mathcal{N}_{\mathbb{C}}(s_i; r_n^{(t)}, \Sigma_n^{(t)}) p(s_i)}{\sum_{s_i \in \mathcal{S}} \mathcal{N}_{\mathbb{C}}(s_i; r_n^{(t)}, \Sigma_n^{(t)}) p(s_i)}, \tag{9}$$

$$\hat{v}_n^{(t+1)} = \frac{\sum_{s_i \in \mathcal{S}} |s_i|^2 \mathcal{N}_{\mathbb{C}}(s_i; r_n^{(t)}, \Sigma_n^{(t)}) p(s_i)}{\sum_{s_i \in \mathcal{S}} \mathcal{N}_{\mathbb{C}}(s_i; r_n^{(t)}, \Sigma_n^{(t)}) p(s_i)} - |\hat{x}_n^{(t+1)}|^2. \tag{10}$$

As can be observed in **Algorithm 1**, the performance of the AMP algorithm is mainly determined by the accuracy of the equivalent AWGN model (33). In [11], it was shown that the equivalent AWGN model is asymptotically accurate when the dimensions of the system tend to infinity, i.e., $M, N \to \infty$. However, in practical finite-dimensional systems, the performance of the AMP algorithms is far from optimal and even has an error floor owing to the inaccurate assumption, which motivates us to improve the AMP algorithm with the advanced DL technique, i.e., GNNs.

C. MPNN Module

GNNs have been recently adopted for wireless communications as they can incorporate the graph topology of the wireless network into the neural network design and optimize the objectives with the data-driven methods [36]. In particular, the antennas or users are considered as the nodes while the channels are considered as the edges to construct the GNNs. In our proposed AMP-GNN framework, we adopt the MPNN [33]. The MPNN, as illustrated in Fig. 3, consists of L cascade layers, where each node n is connected to all other nodes. The aggregation module, comprising GRU module U and a linear network, is utilized by each node to update the node hidden vector $\mathbf{u}_n^{(l)}$. Note that each node n shares the same weights of the GRU module U and linear network. Furthermore, a message $m_{jn}^{(l)}$ must be calculated for each pair of variable nodes n and j during the l-th round of the MPNN. Specifically, the message $m_{jn}^{(l)}$ is obtained using the multi-layer perceptron (MLP) module D, with each pair n and j sharing the same weights. The reason to adopt the MPNN is that it can unify various GNNs and graph convolutional network approaches [34]. Furthermore, the propagation and aggregation modules in MPNN are very similar to the message passing operation on the factor graph. As a result, its structure is very suitable for statistical inference problems. As mentioned in Section III-B, the AMP algorithm can decouple the linear mixing model (1) into the equivalent AWGN model (33). However, the decoupling is not accurate enough which means that the N equivalent AWGN channels are not independent; i.e., there exists structured dependency among these nodes[4]. This motivates us to adopt MPNN to exploit the correlation between the equivalent AWGN channels and mitigate interference.

In the literature of machine learning, the pair-wise MRF has been utilized to model the structured dependency of a set of random variables $\mathbf{x} = \{x_1, \cdots, x_N\}$ by an undirected graph $G = \{V, E\}$. It can be adopted to model the correlation of the nodes in GNNs. Specifically, the n-th variable node is characterized by a self potential $\phi(x_n)$, and the (n, j)-th pair of the edge is characterized by a pair potential $\psi(x_n, x_j)$, which are given by

$$\phi(x_n) = \exp\left(\frac{1}{\sigma^2}\mathbf{y}^T\mathbf{a}_n x_n - \frac{1}{2}\mathbf{a}_n^T\mathbf{a}_n x_n^2\right)\mathsf{P}(x_n), \tag{11a}$$

[4]As demonstrated in [45], if the asymptotic conditions of AMP are satisfied, the structured dependency of the nodes is eliminated due to the central limit theorem (CLT). However, this conclusion is highly dependent on the large-system system limit. In finite-dimensional systems, the structured dependency of nodes still exists. Additionally, for high-order modulation symbols (e.g.,16-QAM and 64-QAM), there is an SER floor when $M/N > \beta_{th}$, indicating that the structured dependency of nodes cannot be ignored in these cases [46].

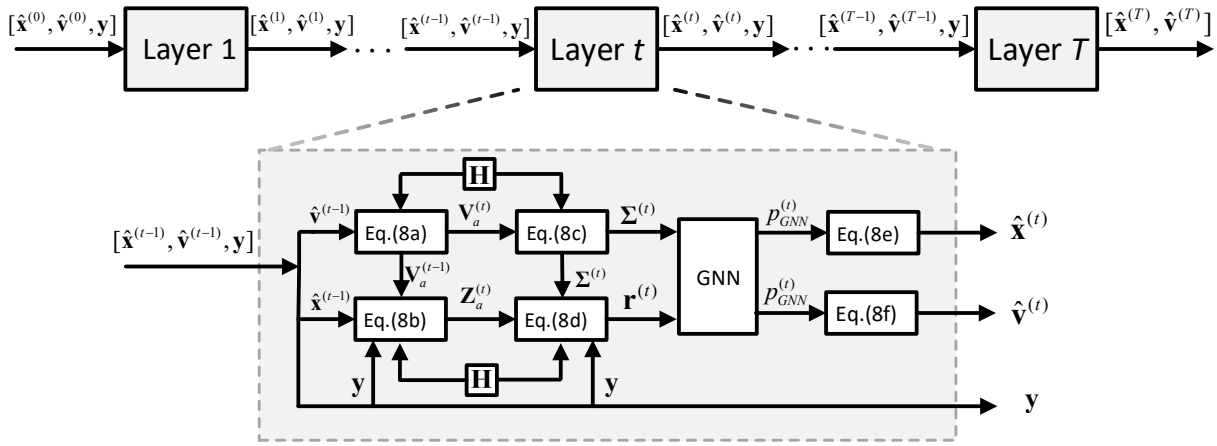


Fig. 2. The structure of the proposed AMP-GNN network.

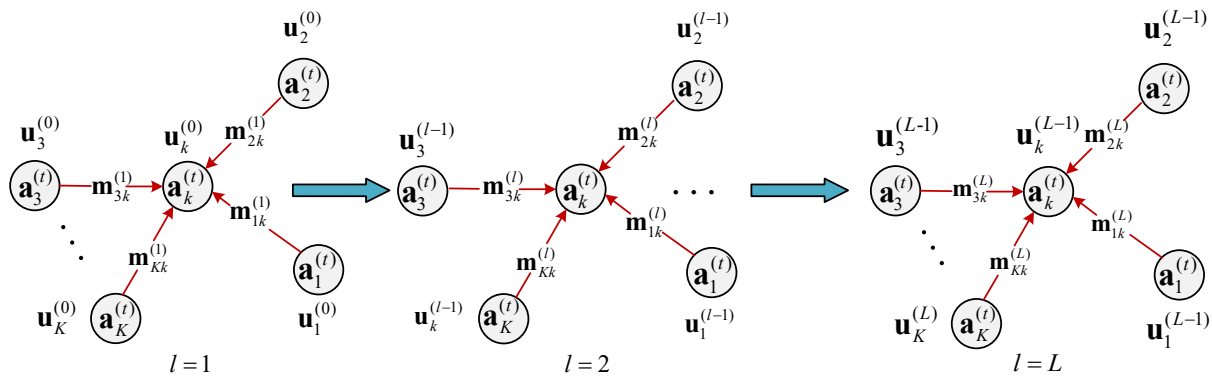


Fig. 3. The block diagram of the structure for MPNN.

$$\psi(x_n, x_j) = \exp\left(-\frac{1}{\sigma^2}\mathbf{a}_n^T\mathbf{a}_j x_n x_j\right), n, j \in 1, \ldots, N, \quad \text{and} \quad n \neq j \tag{11b}$$

respectively, where $\mathbf{a}_n$ denotes the n-th column of the matrix $\mathbf{A}$. The posterior probability $p_{\text{GNN}}(\mathbf{x}|\mathbf{y})$ corresponding to the pair-wise MRF for the statistical inference problem can be obtained by the MPNN and written as [37]

$$p_{\text{GNN}}(\mathbf{x}|\mathbf{y}) = \frac{1}{Z}\prod_{n=1}^{N}\phi(x_n)\prod_{\substack{j=1\\ j\neq n}}^{N}\psi(x_n, x_j), \tag{12}$$

where Z is a normalization constant. In particular, $\phi(x_n)$ and $\psi(x_n, x_j)$ can be represented by the information of the nodes and edges in GNNs, respectively. Equation (12) implies that a well-trained MPNN can characterize the posterior probability $p_{\text{GNN}}(\mathbf{x}|\mathbf{y})$, which is the key of statistical inference.

The MPNN adopted in the AMP-GNN is composed of three main modules: a *propagation module*, an *aggregation module*, and a *readout module*. The first two modules operate in all layers while the readout module is involved only after the last layer. To better understand the structure of the MPNN, we first elaborate on the following concepts about the GNNs. In particular, we mainly introduce the definitions of node and edge and their associated attributes.

- **Node:** In GNNs, each node $n \in V$ represents the n-th user or n-th antenna in the wireless systems.
- **Node Attribute:** Each node has an assigned node attribute $\mathbf{a}_n$ that is constant when exchanging information between different nodes. In the proposed AMP-GNN, the MPNN in the t-layer takes the output from the linear module of the AMP algorithm as a node attribute.
- **Edge:** An edge $e_{n,j} \in E$ is to connect node $n \in V$ and $j \in V$. Whether the edge exists or not depends on the graph structure for the target problem.
- **Edge Attributes:** Each edge $e_{n,j}$ has an assigned edge attribute $\mathbf{f}_{jn}$ that is constant when computing the message. In the proposed AMP-GNN, the MPNN uses the CSI and noise level as the edge attributes.
- **Hidden vector:** Each node n has a hidden vector $\mathbf{u}_n$ updated in different rounds of the MPNN, and will be used to compute the output of the GNN.
- **Message:** The incoming messages $\mathbf{m}_{jn}$ from its connected edges are utilized to update the node feature vector $\mathbf{u}_n$.

When designing GNNs, we need to first define the node and edge attributes. As the MPNN in the t-layer of the AMP-GNN takes the output from the linear module in the AMP algorithm as the input, it is natural to incorporate the mean $r_n^{(t)}$ and variance $\Sigma_n^{(t)}$ obtained from (8c) and (8d) into the attribute $\mathbf{d}_n^{(t)}$ of the variable node x_n by concatenating the

mean and variance as

$$\mathbf{d}_n^{(t)} = \left[r_n^{(t)}, \Sigma_n^{(t)}\right]. \tag{13}$$

The second step is to define the initialized hidden vector $\mathbf{u}_n^{(l)}$ for each node x_n. We consider the initial value calculated from encoding the information of the received signal $\mathbf{y}$, corresponding linear vector $\mathbf{d}_n^{(t)}$, and noise variance σ^2. The encoding process is implemented by using a single layer neural network given by

$$\mathbf{u}_n^{(0)} = \mathbf{W}_1 \cdot [\mathbf{y}^T\mathbf{a}_n, \mathbf{a}_n^T\mathbf{a}_n, \sigma^2]^T + \mathbf{b}_1, \tag{14}$$

where $\mathbf{W}_1 \in \mathbb{R}^{N_u \times 3}$ is a learnable matrix, $\mathbf{b}_1 \in \mathbb{R}^{N_u}$ is a learnable vector, and N_u is the size of the hidden vector. The edge attribute $\mathbf{f}_{jn} \triangleq \left[\mathbf{a}_n^T\mathbf{a}_j, \sigma^2\right]$ is obtained by extracting the pair potential information from (11b) and is then utilized for the message passing of the MPNN. Based on these definitions and operations, we elaborate on the details of each module in the MPNN in the following, including the propagation, aggregation, and readout modules.

1) Propagation module: For any pair of variable nodes x_n and x_j in (33), we assume that there is an edge to connect them. In the l-th round of the MPNN, each edge first concatenates the hidden vectors $\mathbf{u}_n^{(l-1)}$ and $\mathbf{u}_j^{(l-1)}$ with its own edge attribute $\mathbf{f}_{jn}$ as

$$\mathbf{c}_n^{(l-1)} = [\mathbf{u}_n^{(l-1)}, \mathbf{u}_j^{(l-1)}, \mathbf{f}_{jn}]. \tag{15}$$

Then, it uses the concatenated features $\mathbf{c}_n^{(l-1)}$ as the input for the multi-layer perceptron (MLP). Therefore, the output of the MLP is given by

$$\mathbf{m}_{jn}^{(l)} = \mathsf{D}\left(\mathbf{c}_n^{(l-1)}\right), \tag{16}$$

where D is the MLP operation. In the propagation module, each edge has an MLP with two hidden layers of sizes N_{h_1} and N_{h_2} and an output layer of size N_u. Furthermore, the rectifier linear unit (ReLU) activation function is used at the output of each hidden layer. Finally, the outputs $\mathbf{m}_{jn}^{(l)}$ are fed back to the nodes as shown in Fig. 3. In particular, $\mathbf{m}_{jn}^{(l)}$ can be interpreted as the message transmitted from node j to node n.

2) Aggregation module: The n-th variable node sums all the incoming messages $\mathbf{m}_{jn}^{(l)}$ from its connected edges and concatenates the sum of the $\mathbf{m}_{jn}^{(l)}$ with the t-layer node attribute $\mathbf{d}_n^{(t)}$ as $\mathbf{m}_n^{(l)} = \left[\sum_{\substack{j=1 \\ j\neq n}}^{N} \mathbf{m}_{jn}^{(l)}, \mathbf{d}_n^{(t)}\right]$. Then, the message $\mathbf{m}_n^{(l)}$ is used to compute the node hidden vector $\mathbf{u}_n^{(l)}$ as

$$\mathbf{g}_n^{(l)} = \mathsf{U}\left(\mathbf{g}_n^{(l-1)}, \mathbf{m}_n^{(l)}\right), \tag{17a}$$

$$\mathbf{u}_n^{(l)} = \mathbf{W}_2 \cdot \mathbf{g}_n^{(l)} + \mathbf{b}_2, \tag{17b}$$

where the function U is specified by the gated recurrent unit (GRU) network, whose current and previous hidden states are $\mathbf{g}_n^{(l)} \in \mathbb{R}^{N_{h_1}}$ and $\mathbf{g}_n^{(l-1)} \in \mathbb{R}^{N_{h_1}}$, respectively. $\mathbf{g}_n^{(l)}$ can be interpreted as the intermediate variables to update the hidden vector $\mathbf{u}_n^{(l)}$. In (17b), $\mathbf{W}_2 \in \mathbb{R}^{N_u \times N_{h_1}}$ is a learnable matrix, and $\mathbf{b}_2 \in \mathbb{R}^{N_u}$ is a learnable vector. The updated feature vector $\mathbf{u}_n^{(l)}$ is then sent to the propagation module for the next iteration.

3) Readout module: After L rounds of the message passing between the propagation and aggregation modules, a readout module R is utilized to output the estimated result. We need to especially design the final results from the MPNN for the next AMP-GNN iteration. Here we take the classification problem as an example. Specifically, we consider a readout module that is utilized in each node to output the final estimated distribution $\mathsf{p}_{\text{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$ for the t-layer of the AMP-GNN and is given by[5]

$$\mathsf{p}_{\text{GNN}}^{(t)}(x_n|\mathbf{y}) = \mathsf{R}\left(\mathbf{u}_n^{(L)}\right), \tag{18}$$

The readout function R consists of an MLP with two hidden layers of sizes N_{h_1} and N_{h_2}, and ReLU activation is utilized at the output of each hidden layer. Finally, the hidden and node hidden vectors are updated as

$$\mathbf{g}_n^{(0)} \leftarrow \mathbf{g}_n^{(L)} \text{ and } \mathbf{u}_n^{(0)} \leftarrow \mathbf{u}_n^{(L)}, \quad n = 1, \ldots, N, \tag{19}$$

for the MPNN initialization in the next AMP-GNN iteration. The obtained distribution $\mathsf{p}_{\text{GNN}}^{(t)}(x_n|\mathbf{y})$ will be passed to the nonlinear module in the AMP to further refine the estimated result.

To better understand the output of the MPNN, we take the massive MIMO detection as an example. As the transmitted signal is drawn from the discrete Q-QAM set, we further refine the $\mathsf{p}_{\text{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$ with prior information $\mathsf{P}(x_n)$ and compute the posterior mean and variance for the next layer of the AMP-GNN, which are given by

$$\hat{x}_n^{(t+1)} = \mathbb{E}\{x_n|\mathsf{p}_{\text{GNN}}^{(t)}(x_n|\mathbf{y})\}, \tag{20a}$$

$$\hat{v}_n^{(t+1)} = \text{Var}\{x_n|\mathsf{p}_{\text{GNN}}^{(t)}(x_n|\mathbf{y})\}. \tag{20b}$$

After computing (20a) and (20b), the posterior mean $\hat{x}_n^{(t+1)}$ and $\hat{v}_n^{(t+1)}$ are used for the next AMP-GNN iteration. Finally, the AMP-GNN is executed iteratively until terminated by a fixed number of layers. Note that the expectation and variance in (20a) and (20b) are computed with respect to $\mathsf{p}_{\text{GNN}}^{(t)}(x_n|\mathbf{y})$. This is the main difference between AMP and AMP-GNN. In particular, the $\mathsf{p}_{\text{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$ is assumed to be Gaussian pdf and obtained by the equivalent AWGN model. By contrast, $\mathsf{p}_{\text{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$ is obtained by learning from the data and is not the Gaussian pdf in the AMP-GNN. Thus, the inaccurate Gaussian pdf is refined by the GNNs.

D. Properties of AMP-GNN

The AMP-GNN enjoys several properties that are favorable to solve statistical inference problems in wireless communications, including permutation equivariance and generalization to different numbers of users. These advantages enable the AMP-GNN to learn more efficiently, avoid over-fitting, and develop strong generalizability.

[5]Here we assume the MPNN is utilized for classification problems, thus the output of the Readout module is a discrete distribution $\mathsf{p}_{\text{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$. We can also obtain the continuous value of the output for regression problems.

1) Permutation Equivariance: The first characteristic is permutation equivariance. Consider a set $\mathcal{F}$ of all functions $f : \mathbb{R}^{M\times M} \to \mathbb{R}^M$ and a generic permutation matrix $\mathbf{\Pi} \in \{0,1\}^{M\times M}$, we have the following definition of permutation equivariance.

Definition 1. A function $f \in \mathcal{F}$ is permutation equivariant if $\mathbf{\Pi}^T f(\mathbf{A}, \mathbf{y}) = f(\mathbf{A\Pi}, \mathbf{y})$ for all matrices and all permutations $\mathbf{\Pi}$. In particular, if we permute the labels of the users or antennas in our network before computing the permutation equivariant function f, the individual output values are not changed but only permuted by this same permutation. Note that this is especially critical for our problem because the index of the node is arbitrary and should not play any role in the estimated result.

Proposition 1. Assuming that $\mathbf{\Pi}$ is a permutation matrix, $\tilde{\mathbf{A}} = \mathbf{A\Pi}$ denotes the permuted channel matrix, and $\tilde{\mathbf{x}} = \mathbf{\Pi}^T\mathbf{x}$ is the permutated signal vector. We have following permutated $\tilde{\mathbf{A}}$ and $\tilde{\mathbf{x}}$

$$\mathbf{y} = \tilde{\mathbf{A}}\tilde{\mathbf{x}} + \mathbf{n} = \mathbf{A\Pi\Pi}^T\mathbf{x} + \mathbf{n} = \mathbf{A}\mathbf{x} + \mathbf{n}. \tag{21}$$

Thus, we have

$$\mathbf{\Pi}^T f_\theta(\mathbf{A}, \mathbf{y}) = f_\theta(\mathbf{A\Pi}, \mathbf{y}), \tag{22}$$

for the proposed AMP-GNN.

Proof: Refer to Appendix A.

Permutation equivariance implies that the ordering of the users will not affect network performance. This is because reordering the users simply permutes the columns of the matrix $\mathbf{A}$ and is associated with an appropriate permutation of the symbol vector $\mathbf{x}$. Furthermore, it also reduces the training sample complexity and training time compared to conventional MLPs and CNNs. This is because for each training sample, all its permutations are naturally contained in the training set. But for MLPs and CNNs, data argumentation is required to achieve the same performance. Thus, permutation equivariance is extremely helpful in reducing the training sample complexity and time compared to MLPs and CNNs.

2) Generalize to Different Numbers of Users: Most of the works on DL-based physical layer processing are trained and tested with a fixed number of antennas. However, the number of users (antennas) in practical massive MIMO systems may quickly change. For example, the number of active users is continuously changing with the dynamic nature of wireless networks. Training multiple networks with each network targeted to a specific number of users is not practical. In essence, constructing a network that can handle a varying number of users requires the network to be modular while sharing the same set of parameters for each user. In the AMP-GNN, the AMP algorithm is unrelated to the number of nodes and the dimension of the adopted MLP in MPNN is invariant with the number of users. Furthermore, they share the same parameters for different edges and users. As a result, the proposed AMP-GNN has the ability to handle a varying number of users with a single model and we can train the AMP-GNN with a specific dimension and apply them to different settings.

E. Why AMP-GNN can Enhance AMP?

In Section II-B, we have elaborated on the derivation of the BP algorithm for Bayesian inference. However, the message updates in (4b) involve a high dimensional integral and true pdf, which have extremely high complexity. To this end, the AMP algorithm is derived by exploiting the central limit theorem and Taylor expansion to simplify the BP algorithm. On the other hand, the AMP algorithm can be derived alternatively from the perspective of EP [47]. In particular, the BP update in (4b) can be approximated by

$$\mu_{n\to m}^{(t+1)}(x_n) \propto \frac{\mathrm{Proj}[\mathsf{P}(x_n)\prod_b \mu_{b\to n}^{(t)}(x_n)]}{\mu_{m\to n}^{(t)}(x_n)}, \tag{23}$$

$$\mu_{m\to n}^{(t)}(x_n) \propto \int \frac{\mathrm{Proj}[\mathsf{P}(y_m|\mathbf{x})\prod_{j\neq i}^{N} \mu_{j\to m}^{(t)}(x_j)\mathrm{d}\mathbf{x}_{\backslash n}]}{\mu_{n\to m}^{(t)}(x_n)}, \tag{24}$$

where $\mathrm{Proj}[\mathsf{p}]$ is the projection of a distribution p to a distribution set $\mathcal{F}$ defined as

$$\mathrm{Proj}_{\mathcal{F}}[\mathsf{p}] = \arg\min_{q\in\mathcal{F}} D(\mathsf{p}\|\mathsf{q}), \tag{25}$$

and $D(\mathsf{p}\|\mathsf{q})$ denotes the Kullback-Leibler divergence. To reduce the overhead of transmitting the message, $\mathcal{F}$ is assumed to be the Gaussian distribution. This is because the Gaussian distribution can be fully characterized by its mean and variance, and thus only the mean and variance need to be calculated and passed. First, we assume that $\mu_{n\to m}^{(t)}(x_n) \propto \mathcal{N}_{\mathbb{C}}(x_j; \hat{x}_{j\to m}^t, v_{j\to m}^t)$, and therefore $\prod_b \mu_{b\to n}^{(t)}(x_n)$ is the product of Gaussian distributions. Consider $y_m = a_{mn}x_n + \sum_{j\neq n} a_{mj}x_j + n_m$ and define $Z_{mn} = \sum_{j\neq n} |a_{mn}|^2 v_{j\to m}^t \sim \mathcal{N}_{\mathbb{C}}(x; Z_{m\to n}^t, V_{m\to n}^t)$, we have

$$Z_{m\to n}^t = \sum_{j\neq n} a_{mj}x_{j\to m}^t, \quad \text{and} \quad V_{m\to n}^t = \sum_{j\neq n} |a_{mj}|^2 v_{j\to m}^t. \tag{26a}$$

Then, we obtain $\int[\mathsf{P}(y_m|\mathbf{x})\prod_{j\neq i}^{N}\mu_{j\to m}^{(t)}(x_j)\mathrm{d}\mathbf{x}_{\backslash n}] \propto \mathcal{N}_{\mathbb{C}}\left(x_n; \frac{y_m - Z_{m\to n}^t}{a_{mn}}, \frac{\sigma^2 + V_{m\to n}^t}{|a_{mn}|^2}\right)$. As a result, we have $\mu_{m\to n}^{(t)}(x_n) \propto \mathcal{N}_{\mathbb{C}}\left(x_n; \frac{y_m - Z_{m\to n}^t}{a_{mn}}, \frac{\sigma^2 + V_{m\to n}^t}{|a_{mn}|^2}\right)$. By adopting the Gaussian product lemma[6], we have $\prod_b \mu_{b\to n}^{(t)}(x_n) \propto \mathcal{N}_{\mathbb{C}}(x_n; r_n^t, \Sigma_n^t)$, where

$$\Sigma_n^t = \left(\sum_m \frac{|a_{mn}|^2}{\sigma^2 + V_{m\to n}^t}\right)^{-1}, \tag{27a}$$

$$\mu_n^t = \Sigma_n^t\left(\sum_m \frac{h_{mn}^*(y_m - Z_{m\to n}^t)}{\sigma^2 + V_{m\to n}^t}\right). \tag{27b}$$

[6]In the derivation, we use the result that the product of multiple Gaussian distributed random variables yields another Gaussian random variable [48]. Take the product of two Gaussian as an example, we have $\mathcal{N}(x|a, A)\mathcal{N}(x|b, B) = Z\mathcal{N}(x|c, C)$, where $c = C(A^{-1}a + B^{-1}b)$, $C = (A^{-1} + B^{-1})^{-1}$, and $Z = \frac{1}{\sqrt{2\pi(A+B)}}\exp\left(-\frac{(a-b)^2}{2(A+B)}\right)$.

TABLE III: Computational complexity (the number of multiplications) of different detectors.

MIMO settings \ Detectors	OAMP	GNN	GEPNet	AMP	AMP-GNN	EP
64×64	8.22×10^6	1.17×10^6	5.11×10^6	1.78×10^5	2.35×10^6	2.93×10^6
256×256	5.21×10^8	1.27×10^7	2.02×10^9	2.68×10^6	1.93×10^7	1.85×10^9
1024×1024	3.33×10^{10}	5.56×10^8	1.24×10^{10}	4.22×10^7	6.14×10^8	1.18×10^{10}

According to (23), we have $\mathrm{Proj}[\mathsf{P}(x_n)\prod_b \mu_{b\to n}^{(t)}(x_n)] \propto \mathcal{N}_{\mathbb{C}}(x_n;\hat{x}_n^{t+1},\hat{v}_n^{t+1})$ and

$$\hat{x}_n^{(t+1)} = \mathbb{E}\{x_n|r_n^{(t)},\Sigma_n^{(t)}\}, \tag{28a}$$

$$\hat{v}_n^{(t+1)} = \mathrm{Var}\{x_n|r_n^{(t)},\Sigma_n^{(t)}\}. \tag{28b}$$

As the Proj operation utilizes a Gaussian distribution to approximate $\mathrm{Proj}[\mathsf{P}(x_n)\prod_b \mu_{b\to n}^{(t)}(x_n)]$, its accuracy depends on the prior information $\mathsf{P}(x_n)$ and the approximated message $\prod_b \mu_{b\to n}^{(t)}(x_n)$. However, the approximated message $\prod_b \mu_{b\to n}^{(t)}(x_n)$ is far from the Gaussian pdf and $\mathsf{P}(x_n)$ is not exactly known for some applications in practical massive MIMO systems [7]. To this end, GNNs can learn an accurate Proj from the data beyond the specific distribution $\mathsf{P}(x_n)$ and inaccurate approximation $\prod_b \mu_{b\to n}^{(t)}(x_n)$ by its powerful capability, which is the underlying reason why the AMP-GNN outperforms the AMP algorithm. In other words, GNNs can learn an accurate approximation for the Kullback-Leibler divergence.

IV. Application to Massive/Ultra-Massive MIMO Systems

As mentioned in Section II-A, abundant applications in wireless communications can be categorized into statistical inference problems. Although we have provided a general framework, some dedicated network design should be considered for specific applications. To show the effectiveness of the AMP-GNN framework, we take the massive/ultra-massive MIMO detection as an example in this section. We first elaborate the application of AMP-GNN for massive/ultra-massive MIMO detection. Then, the computational complexity of the proposed AMP-GNN-based MIMO detector is analyzed.

A. Massive/Ultra-Massive MIMO Detection

In massive and ultra-massive MIMO systems, the dimension of antenna arrays is extremely large. Efficient MIMO detection algorithms, which balance performance and complexity, are of significant importance to fully unleash the potential of such large-scale systems. We consider an uplink multi-user MIMO (MU-MIMO) systems where the base station (BS) equipped with M antennas serves N single-antenna users. Assuming that the symbol vector $\mathbf{x} \in \mathbb{C}^{M\times 1}$ is transmitted over a Rayleigh fading channel $\mathbf{H} \in \mathbb{C}^{M\times N}$ and each element of $\mathbf{H}$ and $\mathbf{x}$ is drawn from an independent and identically distributed (i.i.d.) complex Gaussian distribution and a Q-QAM constellation, respectively. Thus, the received signal $\mathbf{y} \in \mathbb{C}^{M\times 1}$ is given by $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$.

One challenge of applying the AMP-GNN framework to MIMO detection is how to design the readout module. As the MIMO detection is a classification problem, the output size of R is the cardinality of the real-valued constellation set, i.e., $\sqrt{Q}$. We further use the softmax function to restrict the output of each node in a probabilistic form,

$$\tilde{\mathsf{p}}_{\mathrm{GNN}}^{(t)}(x_n = s_i|\mathbf{y}) = \frac{\exp\left(\mathsf{p}_{GNN}^{(t)}(x_n = s_i|\mathbf{y})\right)}{\sum_{s_i\in\mathcal{S}} \exp\left(\mathsf{p}_{\mathrm{GNN}}^{(t)}(x_n = s_i|\mathbf{y})\right)}, \quad s_i \in \mathcal{S}. \tag{29}$$

As the distribution of the transmitted signal is known (i.e., Q-QAM), we further refine the $\mathsf{p}_{\mathrm{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$ with prior information $\mathsf{P}(x_n)$ and compute the posterior mean and variance for the next layer of the AMP-GNN, which are given by

$$\hat{x}_n^{(t+1)} = \mathbb{E}\{x_n|\tilde{\mathsf{p}}_{\mathrm{GNN}}^{(t)}(x_n = s_i|\mathbf{y})\}, \tag{30a}$$

$$\hat{v}_n^{(t+1)} = \mathrm{Var}\{x_n|\tilde{\mathsf{p}}_{\mathrm{GNN}}^{(t)}(x_n = s_i|\mathbf{y})\}, \tag{30b}$$

where the expectation and variance are computed with respect to $\tilde{\mathsf{p}}_{\mathrm{GNN}}^{(t)}(x_n = s_i|\mathbf{y})$.

B. Complexity Analysis

In this section, we analyze the computational complexity of the AMP-GNN-based MIMO detector and compare it with existing DL and message passing based MIMO detectors. Specifically, the complexity of the AMP detector is $\mathcal{O}(MN)$ due to the matrix-vector multiplication while the complexity for MPNN is $\mathcal{O}(NN_{h1}N_{h2})$ which accounts for the MLP operation. Therefore, the computational complexity of the AMP-GNN is $\mathcal{O}(MN + NN_{h1}N_{h2})$, dominated by the complexity of the AMP and MPNN. In contrast, the complexity of the GEPNet is $\mathcal{O}(MN^2 + NN_{h1}N_{h2})$ which includes the computational complexity of the EP and GNN.

To conduct a fair comparison of the computational complexity, we will use the number of multiplications as the metric and show the exact values for different MIMO settings with quadrature phase shift keying (QPSK) symbols in Table III. Compared with the state-of-the-art DL-based MIMO detectors, i.e., GEPNet, the AMP-GNN entails a much lower complexity. In particular, the ratio between the complexity of the AMP-GNN and GEPNet is dramatically reduced when the number

[7]The classical AMP algorithm proved Gaussianity based on the GLT in the large-system limit [45]. However, in practical finite-dimensional MIMO systems, Gaussianity cannot be guaranteed. As emphasized in [46], the statistics of $\mathbf{r}^{(t)}$ are not Gaussian and therefore cannot be accurately tracked by state evolution, which incurs the performance loss. Additionally, as analyzed in [46], there are parameter regimes where the AMP-based MIMO detection only achieves suboptimal performance even in the large-system limit. In the cases where the performance of the AMP is far from optimal, GNN can be used for performance enhancement.

TABLE IV: Simulation parameters of the AMP-GNN for massive MIMO detection.

Simulation parameters	Value
Number of users (N)	16, 24, 32, 64
Number of antennas (M)	16, 24, 32, 64
Number of realizations (d)	100000
The hyperparameters for MPNN	$N_{h_1} = 16$, $N_{h_2} = 8$, and $N_u = 8$
Modulation symbols (Q-QAM)	4-QAM, 16-QAM, 64-QAM
Training SNR	SNR = 20 dB

of users increases. For example, the ratio between the complexity of the AMP-GNN and GEPNet is only 45.99% when $M = N = 64$ while the ratio is significantly reduced to 4.95% when $M = N = 1024$. This is because the complexity of matrix inversion in the GEPNet is the dominant term, which is prohibitively high when the number of antennas and user is large. In contrast, the AMP-GNN only involves matrix-vector multiplications, which is a favorable feature for future ultra-massive MIMO systems. On the other hand, the propagation and aggregation modules on each node and edge are executed in parallel, which means the MPNN can be further processed in a distributed manner and the time complexity can also be reduced. This is also a great advantage for distributed ultra-massive MIMO systems.

V. Simulation Results

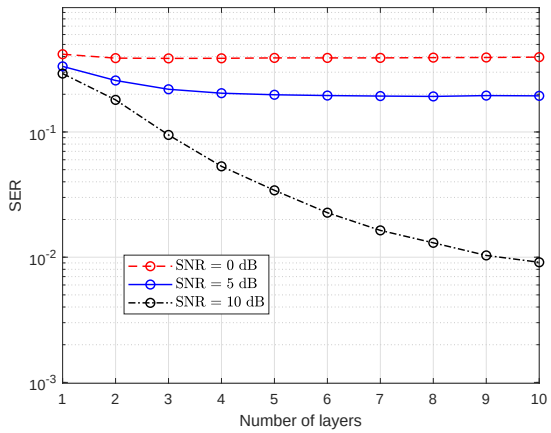
In this section, we mainly provide simulation results of the AMP-GNN for MIMO detection and compare them with other MIMO detectors. We use the symbol error rate (SER) as the performance metric in our simulations. The signal-to-noise (SNR) of the system is defined as $\text{SNR} = \frac{\mathbb{E}\|\mathbf{H}\mathbf{x}\|_2^2}{\mathbb{E}\|\mathbf{n}\|_2^2}$. To illustrate the effectiveness of our proposed AMP-GNN, we adopt several well-established MIMO detectors as baselines:

- **MMSE**: A classical linear receiver for MIMO detection which inverts the received signal by applying the channel-noise regularized pseudo-inverse of the channel matrix.
- **AMP**: An efficient message passing algorithm for MIMO detection given in **Algorithm** 1 and implemented with 10 iterations[8].
- **OAMP-Net**: The OAMP-based model-driven DL detector developed in [27]. Each layer requires computing a matrix pseudo-inverse and has 2 learnable parameters.
- **EP**: The EP-based MIMO detector with 10 iterations as proposed in [14].
- **GEPNet**: The GNN-enhanced EP detector proposed in [40] with 10 layers.

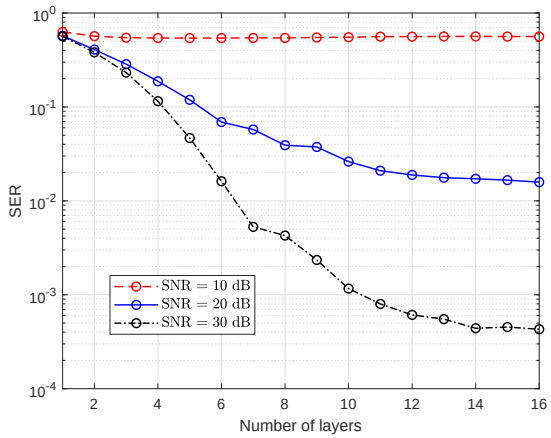
A. Implementation Details

In the simulation, the AMP-GNN is implemented on the PyTorch platform. The number of layers of the AMP-GNN detector is set to $T = 10$ while the number of layers of the GNN is set to $L = 2$. The training data consists of a number of randomly generated pairs $(\mathbf{x}, \mathbf{y})$. The data $\mathbf{x}$ is generated from QAM modulation symbols. We train the network for 500 epochs with the same training and validation sets in each epoch. The training set contains 100,000 samples while the validation set contains 5,000 samples. The AMP-Net is trained using the stochastic gradient descent method and Adam optimizer. The learning rate is set to 0.001 and the batch size is set to 64. We choose L_2 loss as the cost function, which is defined by,

$$L_2(\mathbf{x}, \hat{\mathbf{x}}^{(T)}) = \|\mathbf{x} - \hat{\mathbf{x}}^{(T)}\|^2. \tag{31}$$



(a) 16-QAM.



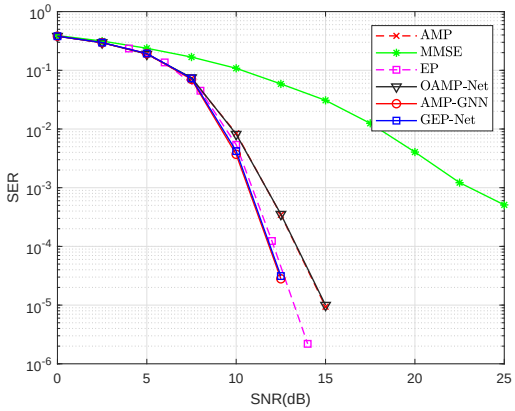
(b) QPSK.

Fig. 4. Convergence analysis of the AMP-GNN versus the number of layers under QPSK and 16-QAM modulation with 32×32 MIMO systems.

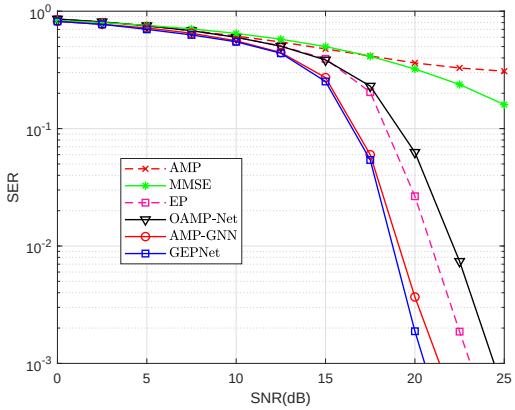
B. Convergence Analysis

First, we analyze the convergence of the AMP-GNN network for MIMO detection. Fig. 4 illustrates the SER performance versus the number of layers under various SNRs with QPSK and 16-QAM symbols. The numbers of antennas and users are $M = N = 32$. As shown in the figure, the AMP-GNN converges within ten layers for all the cases. Furthermore, more numbers of layers are required to be convergent in high SNRs and modulation order. Based on the above observations, we consider the AMP-GNN-based

[8]It was found that a further increase in the number of iterations only offers a negligible performance gain. We set the same number of layers in other DL-based baseline methods for fair comparison.

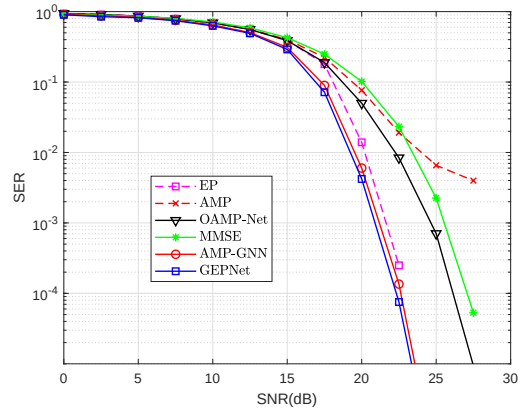


(a) QPSK.

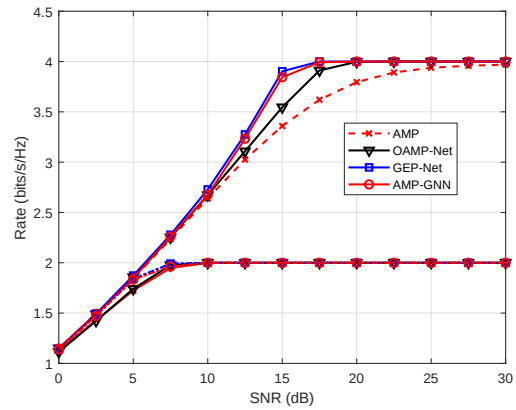


(b) 16-QAM.

Fig. 5. SER comparison of the AMP-GNN with other MIMO detectors under 64×64 Rayleigh MIMO channels with QPSK and 16-QAM symbols.



(a) 64×32 MIMO with 64-QAM.



(b) 32×32 MIMO with QPSK and 16-QAM.

Fig. 6. SER and achievable rate performance of the AMP-GNN with other MIMO detectors.

detectors with ten layers ($T = 10$) in following simulation.

C. Performance Comparison

Fig. 5 compares the average SER of the AMP-GNN with those of the baseline detectors. As can be observed from the figure, the AMP-GNN outperforms almost all MIMO detectors except for the GEPNet detector. In particular, the AMP-GNN outperforms the AMP detector at all SNRs, which demonstrates that the GNN module can improve the AMP detector significantly. Specifically, if we target an SER=10^{-2}, then the performance gain is approximately 3.9 dB compared to the AMP detector. The reason for the performance improvement is that the GNN refine the equivalent AWGN model with a more accurate distribution $\mathsf{p}^{(t)}_{\text{GNN}}(x_n = s_i|\mathbf{y})$. Furthermore, the AMP-GNN has only 0.8 dB performance loss compared to the GEPNet detector in a 4×4 MIMO system when we target at an SER=10^{-2}. The performance loss then reduces to 0.3 dB for 32×32 MIMO systems as illustrated in Fig. 5. A similar conclusion can be obtained with a higher modulation order in Fig. 6(a). Thus, we conclude that the AMP-GNN has a comparable performance to GEPNet but with a remarkably reduced computational complexity, especially for massive and ultra-massive MIMO systems.

D. Achievable Rates

To provide a clearer performance comparison, we present the achievable rates of different detectors for each user with different modulation symbols. This can be achieved because message passing-based MIMO detectors can decouple the linear mixing model $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$ into a series of independent equivalent AWGN channels given by

$$r_n^{(t)} = x_n + w_n^{(t)}, \tag{32}$$

where $w_n^{(t)} \sim \mathcal{N}_{\mathbb{C}}(w_n^{(t)}; 0, \Sigma_n^{(t)})$. For an arbitrary discrete constellation $\mathcal{S} = \{s_1, \ldots, s_{|\mathcal{S}|}\}$ with equal probability $1/|\mathcal{S}|$, we have the performance of $\text{mmse}(\rho)$ given by,

$$\text{mmse}(\rho) = 1 - \frac{1}{\pi} \int \frac{\sum_{l=1}^{|\mathcal{S}|} s_l e^{-|y-\sqrt{\rho}s_l|^2}}{|\mathcal{S}| \sum_{l=1}^{|\mathcal{S}|} s_l e^{-|y-\sqrt{\rho}s_l|^2}} dy. \tag{33}$$

where $y = \sqrt{\rho}x + z$. Thanks to the relationship between the mutual information and MMSE in the SISO case [49], we have

$$\mathcal{R}_{\mathcal{S}}(\rho^*) = I(x; \sqrt{\rho^*}x + z) = \int_0^{\rho^*} \text{mmse}(\rho) d\rho \tag{34}$$

where $\rho^* = 1/\Sigma_n^{(t)}$. It is shown that the capacity of a SISO-AWGN channel equals to the area under $\mathrm{mmse}(\rho)$ from $\rho = 0$ to $\rho = \rho^*$. As illustrated in Fig. 6(b), the AMP-GNN outperforms OAMP-Net and AMP detectors, and achieves similar performance to GEPNet, which demonstrates that the GNN module can help multi-user interference cancellation.

E. Robustness to Dynamic Numbers of Users

In Fig. 7(a), we train the AMP-GNN in an 32×16 and 32×32 MIMO systems and test it in a 32×24 MIMO system. As shown in the figure, if we target an SER $= 10^{-3}$, then the AMP-GNN still has a 2.0 dB performance gain compared with the conventional AMP detector even when tested with different numbers of users. Furthermore, it has a similar performance as the AMP-GNN trained and tested both in the 32×24 MIMO system, which indicates that the AMP-GNN has strong robustness to different numbers of users in the deployment stage. This is because the GNN has the permutation equivariance property which makes it robust against dynamic changes in the number of users.

F. Robustness to Channel Estimator Error

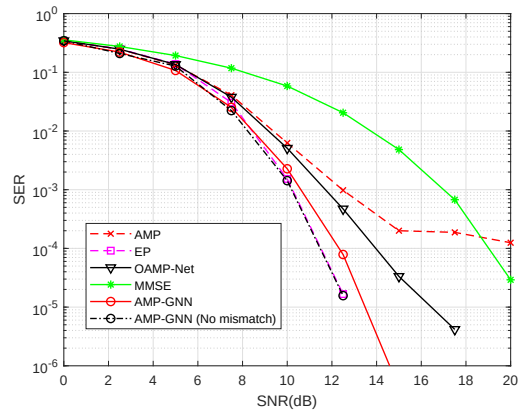
In the aforementioned subsections, we assumed AMP-GNN with perfect CSI. However, channel estimation error normally exists in practical systems even considering high-performance channel estimators. We train the AMP-GNN with perfect CSI and test it with noisy channels, which is given by

$$\hat{\mathbf{H}} = \mathbf{H} + \mathbf{E}. \tag{35}$$

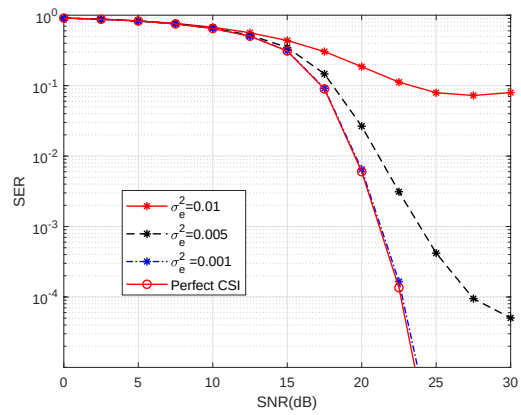
The channel estimator error $\mathbf{E} \sim \mathcal{N}_{\mathbb{C}}(0, \sigma_e^2 \mathbf{I})$ and σ_e^2 denotes the power of the channel estimator error. Fig. 7(b) shows the performance of the AMP-GNN with various powers of channel estimation error. In particular, the performance of the trained AMP-GNN with $\sigma_e^2 = 0.001$ is similar to that with perfect CSI, which demonstrates the AMP-GNN has strong robustness to channel estimator error.

VI. Conclusions

We have developed a novel GNN-enhanced AMP detector for massive/ultra-massive MIMO systems, namely AMP-GNN, which is obtained by incorporating a GNN module into the AMP algorithm. AMP-GNN inherits the low-complexity of the AMP algorithm and high efficiency of the GNN module. It was shown that AMP-GNN improves the performance of the AMP algorithm significantly. Simulation results have also demonstrated that the AMP-GNN-based detector has comparable performance as the state-of-the-art GEPNet detector but with a significantly reduced computational complexity. Furthermore, it is robust to the change in the number of users in practical wireless systems. In the future, it will be interesting to apply the AMP-GNN network to other statistical inference problems in wireless communications, such as channel estimation, beamforming design, localization [50], and nonlinear systems [51].



(a) Robustness to the number of users.



(b) Robustness to channel estimator error.

Fig. 7. Robustness performance of the AMP-GNN to various mismatches.

Acknowledgment

The authors would like to thank Prof. Chao-Kai Wen, from the National Sun Yat-sen University for the discussion of neural enhanced message passing. Furthermore, the authors would like to thank Prof. Wibowo Hardjawana and Dr. Alva Kosasih from The University of Sydney, for sharing the codes for the GEP-Net.

Appendix A
Proof of Proposition 1

To prove *Proposition 1*, we have the following two Propositions.

Proposition 2. If MPNN and AMP modules in the AMP-GNN are permutation equivariant then the AMP-GNN method is also permutation equivariant.

Proof: Due to the transitivity of the permutation equivariance, it suffices to prove each module is equivariant. Therefore, we prove the equivariance for each module separately as follows.

Proposition 3. The AMP algorithm and MPNN are permutation equivariant.

Proof: The AMP algorithm is mainly composed of two modules, the linear and nonlinear modules. The *linear module* is

mainly characterized by Eq.(8a-8d) and the nonlinear module is characterized by Eq.(8e-8f). As the computation process in Eq.(8a-8d) is element-wise, we can obtain $\tilde{\mathbf{\Sigma}}^{(t)} = \mathbf{\Pi}^T \mathbf{\Sigma}^{(t)} \mathbf{\Pi}$ and $\tilde{\mathbf{r}}^{(t)} = \mathbf{\Pi}^T \mathbf{r}^{(t)}$ by substituting $\tilde{\mathbf{H}} = \mathbf{H}\mathbf{\Pi}$. The nonlinear model is also permutation equivariant as it independently performs the denoising for each user. On the other hand, the MPNN has already been proven to be permutation equivariant [35]. Thus, all modules in the proposed AMP-GNN are permutation equivariant. We can hence conclude that the AMP-GNN is permutation equivariant and robust to the user permutations.

REFERENCES

[1] H. He *et al*., "GNN-Enhanced Approximate Message Passing for Massive/Ultra-Massive MIMO Detection," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Glasgow, Scotland, UK, Mar. 2023.

[2] T. L. Marzetta, "Noncooperative cellular wireless with unlimited numbers of base station antennas," *IEEE Trans. Wireless Commun.*, vol. 9, no. 11, pp. 3590–3600, Nov. 2010.

[3] E. Björnson, "A look at an LTE-TDD Massive MIMO product," http://ma-mimo.ellintech.se/2018/08/27/ a-look-at-an-lte-tdd-massive-mimo-product/, accessed: 2019-08-07.

[4] J. G. Andrews, S. Buzzi, W. Choi, S. V. Hanly, A. Lozano, C. K. Soong, and J. C. Zhang, "What will 5G be?", *IEEE J. Sel. Areas Commun.*, vol. 32, no. 6, pp. 1065-1082, Jun. 2014.

[5] K. B. Letaief, W. Chen, Y. Shi, J. Zhang, and Y.-J.-A. Zhang, "The roadmap to 6G: AI empowered wireless networks," *IEEE Commun. Mag.*, vol. 57, no. 8, pp. 84-90, Aug. 2019.

[6] K. B. Letaief, Y. Shi, J. Lu, and J. Lu, "Edge artificial intelligence for 6G: Vision, enabling technologies, and applications," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 5-36, Jan. 2022.

[7] E. Björnson et al.,"Massive MIMO is a reality–What is next? Five promising research directions for antenna arrays," *Digital Signal Process.*, vol. 94, pp. 3-20, Nov. 2019.

[8] E. Biglieri, R. Calderbank, A. Constantinides, A. Goldsmith, A. Paulraj, and H. V. Poor, *MIMO Wireless Communications*. Cambridge, U.K.: Cambridge Univ. Press, 2007.

[9] C. M. Bishop, *Pattern Recognition Machine Learning*. Springer, 2006.

[10] F. R. Kschischange, B. J. Frey, and H. A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 42, no. 2, pp. 498-519, Feb. 2001.

[11] M. Bayati and A. Montanari, "The dynamics of message passing on dense graphs, with applications to compressed sensing," *IEEE Trans. Inform. Theory*, vol. 57, no. 2, pp. 764-785, Feb. 2011.

[12] T. P. Minka, "A family of algorithms for approximate Bayesian Inference," Ph.D. dissertation, Dept. Elect. Eng. Comput. Sci., MIT, Cambridge, MA, USA, 2001.

[13] S. Wu, L. Kuang, Z. Ni, J. Lu, D. Huang, and Q. Guo, "Low-complexity iterative detection for large-scale multiuser MIMO-OFDM systems using approximate message passing," *IEEE J. Sel. Topics Signal Process.*, vol. 8, no. 5, pp. 902-915, Oct. 2014.

[14] J. Céspedes, P. M. Olmos, M. Sánchez-Fern´andez, and F. Pérez-Cruz, "Expectation propagation detection for high-order high-dimensional MIMO systems," *IEEE Trans. Commun.*, vol. 62, no. 8, pp. 2840-2849, Aug. 2014.

[15] C.-K. Wen, S. Jin, K.-K. Wong, J.-C. Chen, and P. Ting, "Channel estimation for massive MIMO using Gaussian-mixture Bayesian learning," *IEEE Trans. Wireless Commun.*, vol. 14, no. 3, pp. 1356-1368, Mar. 2015.

[16] S. Wu, Z. Ni, X. Meng, and L. Kuang, "Block expectation propagation for downlink channel estimation in massive MIMO systems," *IEEE Commun. Lett.*, vol. 20, no. 11, pp. 2225-2228, Nov. 2016.

[17] L. Liu, C. Liang, J. Ma, and L. Ping, "Capacity optimality of AMP in coded systems," *IEEE Trans. Inf. Theory*, vol. 67, no. 7, pp. 4429-4445, Jul. 2021.

[18] J. Ma, L. Liu, X. Yuan and L. Ping, "On orthogonal AMP in coded linear vector systems," *IEEE Trans. Wireless Commun.*, vol. 18, no. 12, pp. 5658-5672, Dec. 2019.

[19] T. O'shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 563-575, Dec. 2017.

[20] Z.-J. Qin, H. Ye, G. Y. Li, and B.-H. Juang, "Deep learning in physical layer communications," *IEEE Wireless Commun.*, vol. 26, no. 2, pp. 93–99, Apr. 2019.

[21] H. He, S. Jin, C.-K. Wen, F. Gao, G. Y. Li, and Z. Xu, "Model-driven deep learning for physical layer communications," *IEEE Wireless Commun.*, vol. 26, no. 5, pp. 77-83, Oct. 2019.

[22] J. Hoydis, F. A. Aoudia, A. Valcarce, and H. Viswanathan, "Toward a 6G AI-native air interface," *IEEE Commun. Mag.*, vol. 59, no. 5, pp. 76-81, May 2021.

[23] H. He, C. K. Wen, S. Jin, and G. Y. Li, "Deep learning-based channel estimation for beamspace mmWave massive MIMO systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 5, pp. 852-855, Oct. 2018.

[24] H. He, R. Wang, W. Jin, S. Jin, C.-K. Wen, and G. Y. Li, "Beamspace channel estimation for wideband millimeter-wave MIMO: A model-driven unsupervised learning approach," *IEEE Trans. Wireless Commun.*, early access, doi: 10.1109/TWC.2022.3206773.

[25] C.-K. Wen, W. T. Shih, and S. Jin, "Deep learning for massive MIMO CSI feedback," *IEEE Wireless Commun. Lett.*, vol. 7, no. 5, pp. 748-751, Oct. 2018.

[26] H. Ye, G. Y. Li, and B.-H. F. Juang, "Power of deep learning for channel estimation and signal detection in OFDM systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 1, pp. 114-117, Feb. 2018.

[27] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Model-driven deep learning for MIMO detection," *IEEE Trans. Signal Process.*, vol. 68, pp. 1702-1715, Mar. 2020.

[28] M. Honkala, D. Korpi, and J. M. Huttunen, "DeepRX: Fully convolutional deep learning receiver," *IEEE Trans. Wireless Commun.*, vol. 20, no. 6, pp. 3925-3940, Jun. 2021.

[29] K. Pratik, B. D. Rao, and M. Welling, "RE-MIMO: Recurrent and permutation equivariant neural MIMO detection," *IEEE Trans. Signal Process.*, vol. 69, pp. 459-473, Jan. 2021.

[30] J. Ma and L. Ping, "Orthogonal AMP," *IEEE Access*, vol. 5, no. 14, pp. 2020-2033, Jan. 2017.

[31] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 119-131, Feb. 2018.

[32] K. Yoon, R. Liao, Y. Xiong, et al., "Inference in probabilistic graphical models by graph neural networks," in *Proc. Asilomar Conf. Signals, Systems, Computers*, Pacific Grove, CA, USA, Nov. 2019, pp. 868-875.

[33] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. Int. Conf. Mach. Learn.*, vol. 70, Aug. 2017, pp. 1263-1272.

[34] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57-81, 2020.

[35] Y. Shen, Y. Shi, J. Zhang, and K. B. Letaief, "Graph neural networks for scalable radio resource management: Architecture design and theoretical analysis," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 101-115, Jan. 2021.

[36] Y. Shen, J. Zhang, S.H. Song, and K. B. Letaief, "Graph neural networks for wireless communications: From theory to practice," *IEEE Trans. Wireless Commun.*, early access, doi: 10.1109/TWC.2022.3219840.

[37] A. Scotti, N. N. Moghadam, D. Liu, K. Gafvert, and J. Huang, "Graph neural networks for massive MIMO detection," in *Proc. Int. Conf. Mach. Learn. (ICML) Workshop, Vienna, Austria*, Jul. 2020.

[38] A. Chowdhury, G. Verma, C. Rao, A. Swami, and S. Segarra, "Unfolding WMMSE using graph neural networks for efficient power allocation," *IEEE Trans. Wireless Commun.*, vol. 20, pp. 6004-6017, Sep. 2021.

[39] V. G. Satorras and M. Welling, "Neural enhanced belief propagation on factor graphs," in *Proc. AISTATS-21*, Apr. 2021, pp. 685-693.

[40] A. Kosasih *et al*., "Graph neural network aided expectation propagation detector for MU-MIMO systems," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 9, p. 2540-2555, Jul. 2022.

[41] X. Zhou, J. Zhang, C.-K. Wen, S. Jin, "Extrinsic graph neural network-aided expectation propagation for turbo-MIMO receiver," in *Proc 2022 Int. Symp. Wireless. Com.Syst. (ISWCS)*, Hangzhou, China, Oct. 2022. pp. 1-6.

[42] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," *IEEE Signal Process. Mag.*, vol. 38, no. 2, pp. 18-44, Mar. 2021.

[43] A. Balatsoukas-Stimming and C. Studer, "Deep unfolding for communications systems: A survey and some new directions," in *Proc. IEEE Int. Workshop Signal Process. Syst. (SiPS)*, Nanjing, China, 2019, pp. 266-271.

[44] S. Ji, Y. Xue, and L. Carin, "Bayesian compressive sensing," *IEEE Trans. Signal Process.*, vol. 56, no. 6, pp. 2346-2356, Jun. 2008.

[45] D. L. Donoho, A. Maleki, and A. Montanari, "Message-passing algorithms for compressed sensing," *Proc. Nat. Acad. Sci.*, vol. 106, no. 45, pp. 18914-18919, 2009.

[46] C. Jeon, R. Ghods, A. Maleki, and C. Studer, "Optimality of large mimo detection via approximate message passing," in *Proc. IEEE Int. Symp. Inf. Theory.*, Jun. 2017, pp. 2333-2337.

[47] X. Meng, S. Wu, L. Kuang, and J. Lu, "An expectation propagation perspective on approximate message passing," *IEEE Signal Process. Lett.*, vol. 22, no. 8, pp. 1194-1197, Aug. 2015.

[48] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning.* Cambridge, MA, USA: MIT Press, 2006.

[49] L. Liu, C. Liang, J. Ma, and L. Ping, "Capacity optimality of AMP in coded systems," *IEEE Trans. Inf. Theory*, vol. 67, no. 7, pp. 4429-4445, Jul. 2021.

[50] M. Liang and F. Meyer, "Neural enhanced belief propagation for cooperative localization," in *Proc. IEEE SSP-21*, Rio de Janeiro, Brazil, Jul. 2021, pp. 326-330.

[51] H. He, C.-K. Wen, and S. Jin, "Bayesian optimal data detector for hybrid mmWave MIMO-OFDM systems with low-resolution ADCs," *IEEE J. Sel. Topics Signal Process.*., vol. 12, no. 3, pp. 469-483, Jun. 2018.